

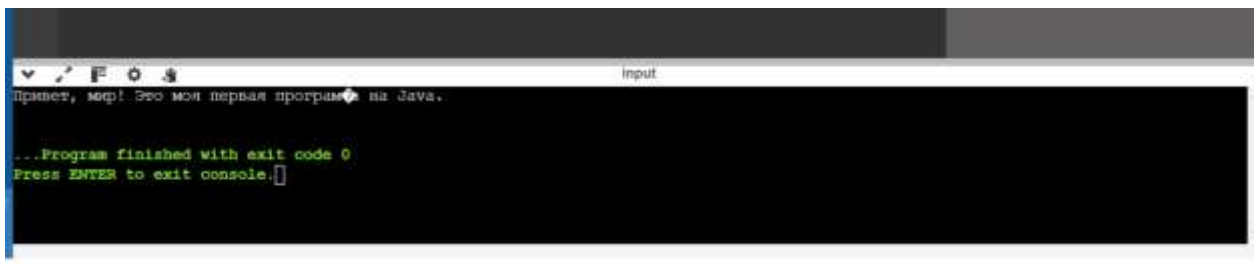
Первая программа на Java: "Hello, World!"	2
ПРАКТИКА №2	2
ПРАКТИКА №3	3
ПРАКТИКА №4	5
ПРАКТИКА №5	7
ПРАКТИКА №6	9
ПРАКТИКА №7	12
ПРАКТИКА №8	15
ПРАКТИКА №9	18
ПРАКТИКА №10	21
ПРАКТИКА №11	24
Практика №12	25
ПРАКТИКА №14	27
ПРАКТИКА №15 и №16	28
ПРАКТИКА №17	30
ПРАКТИКА №18	31

Первая программа на Java: "Hello, World!"

Программа:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Привет, мир! Это моя первая программа на Java.");  
    }  
}
```

Вывод:



ПРАКТИКА №2

Переменные и типы данных

Программа:

```
public class Main {  
    public static void main(String[] args) {  
        // 1. Объявляем переменные  
        String name = "Алексей";  
        int age = 19;  
        double geographyAverageGrade = 4.5;  
        boolean isStudent = true;  
        char gradeLetter = 'B';  
  
        // 2. Выводим информацию на экран  
        System.out.println("Информация о студенте:");  
        System.out.println("Имя: " + name);  
    }  
}
```

```

        System.out.println("Возраст: " + age);

        System.out.println("Средний балл по географии: " + geographyAverageGrade);

        System.out.println("Является студентом: " + isStudent);

        System.out.println("Буквенная оценка: " + gradeLetter);

    }
}

```

Вывод:



ПРАКТИКА №3 **Циклы в Java: for, while, do-while**

Программа:

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // ===== ЗАДАЧА 1: цикл for (числа от 1 до 10) =====
        System.out.println("Задача 1: числа от 1 до 10");
        for (int i = 1; i <= 10; i++) {
            System.out.print(i + " ");
        }

        System.out.println("\n"); // пустая строка для разделения
    }
}

```

```

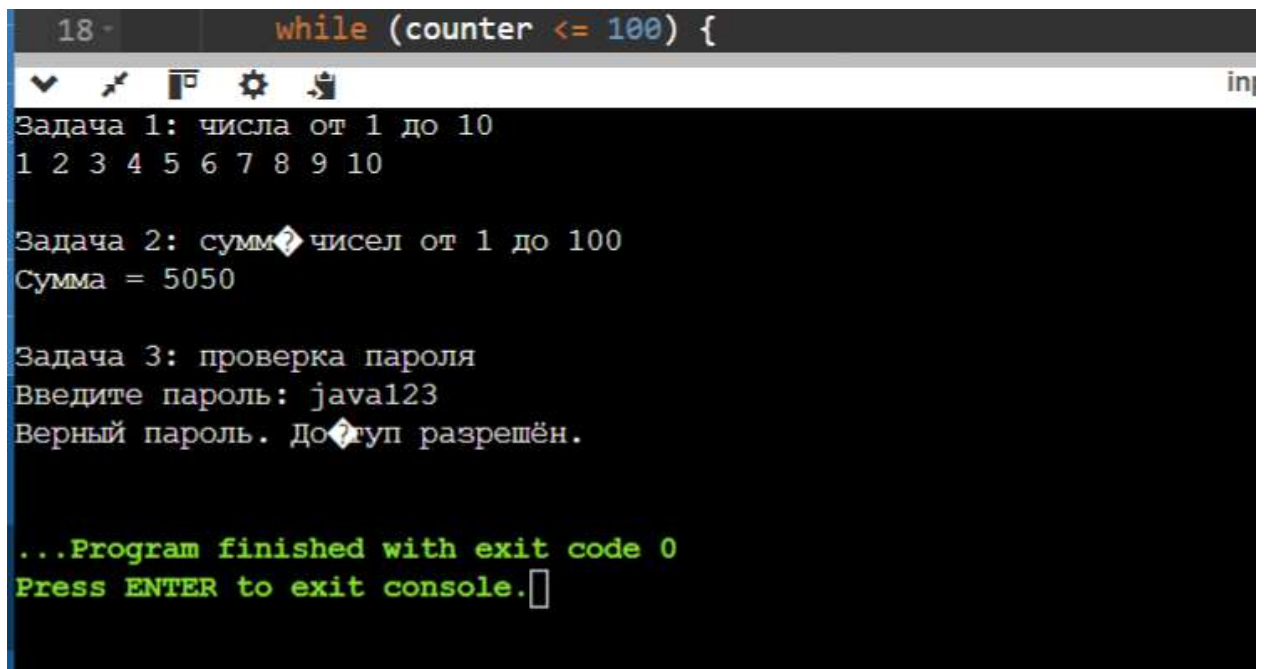
// ===== ЗАДАЧА 2: цикл while (сумма чисел от 1 до 100) =====
System.out.println("Задача 2: сумма чисел от 1 до 100");
int sum = 0;
int counter = 1;
while (counter <= 100) {
    sum = sum + counter; // добавляем текущее число к сумме
    counter++;          // увеличиваем счётчик
}
System.out.println("Сумма = " + sum);
System.out.println(); // пустая строка

// ===== ЗАДАЧА 3: цикл do-while (ввод пароля) =====
System.out.println("Задача 3: проверка пароля");
String password;
do {
    System.out.print("Введите пароль: ");
    password = scanner.next();
    if (password.equals("java123")) {
        System.out.println("Верный пароль. Доступ разрешён.");
        break;
    } else {
        System.out.println("Неверный пароль. Попробуйте ещё раз.");
    }
} while (true);

scanner.close();
}
}

```

Вывод:

A screenshot of a Java IDE's console window. The title bar shows a tab named '18 - while (counter <= 100) {'. The console has a dark background with white and green text. It displays three tasks: Task 1 (printing numbers 1-10), Task 2 (calculating the sum of numbers 1-100 as 5050), and Task 3 (password check for 'java123' which is successful). At the bottom, it shows the program finished with exit code 0 and prompts to press ENTER to exit the console.

```
18 - while (counter <= 100) {  
Задача 1: числа от 1 до 10  
1 2 3 4 5 6 7 8 9 10  
  
Задача 2: сумма чисел от 1 до 100  
Сумма = 5050  
  
Задача 3: проверка пароля  
Введите пароль: java123  
Верный пароль. Доступ разрешён.  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

ПРАКТИКА №4

Массивы: хранение и обработка наборов данных

Программа:

```
public class Main {  
    public static void main(String[] args) {  
  
        // 1. Создать массив из 5 целых чисел  
        int[] numbers = {10, 25, 7, 42, 18};  
  
        // 2. Вывести все элементы массива  
        System.out.println("II. Элементы массива:");  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.print(numbers[i] + " ");  
        }  
    }  
}
```

```
System.out.println("\n");
```

```
// 3. Найти и вывести сумму всех элементов
```

```
int sum = 0;
```

```
for (int i = 0; i < numbers.length; i++) {
```

```
    sum = sum + numbers[i];
```

```
}
```

```
System.out.println("III. Сумма всех элементов: " + sum);
```

```
// 4. Найти и вывести максимальное значение
```

```
int max = numbers[0]; // начинаем с первого элемента
```

```
for (int i = 1; i < numbers.length; i++) {
```

```
    if (numbers[i] > max) {
```

```
        max = numbers[i];
```

```
    }
```

```
}
```

```
System.out.println("IV. Максимальное значение: " + max);
```

```
// 5. Дополнительно: проверить, содержится ли в массиве число 42
```

```
boolean found = false;
```

```
for (int i = 0; i < numbers.length; i++) {
```

```
    if (numbers[i] == 42) {
```

```
        found = true;
```

```
        break;
```

```
    }
```

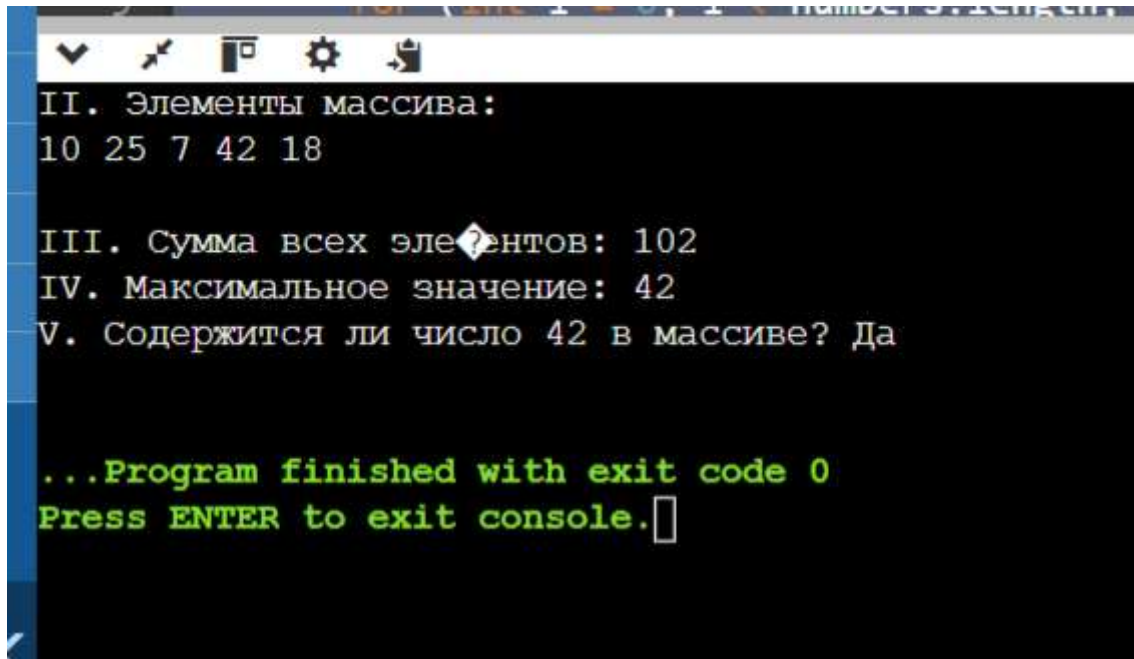
```
}
```

```
System.out.println("V. Содержится ли число 42 в массиве? " + (found ?  
"Да" : "Нет"));
```

```
}
```

}

Вывод:

A screenshot of a console window with a black background and white text. The window has a standard Windows title bar at the top. The output text is as follows:

```
II. Элементы массива:  
10 25 7 42 18  
  
III. Сумма всех элементов: 102  
IV. Максимальное значение: 42  
V. Содержится ли число 42 в массиве? Да  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

ПРАКТИКА №5

Методы: модульность и повторное использование кода

Программа:

```
public class Main {  
  
    // 1. Метод sum(int a, int b) — сумма двух чисел  
    public static int sum(int a, int b) {  
        return a + b;  
    }  
  
    // 2. Перегруженный метод sum(int a, int b, int c) — сумма трёх чисел  
    public static int sum(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // 3. Метод isEven(int number) — проверка на чётность
```

```

public static boolean isEven(int number) {
    return number % 2 == 0;
}

// 4. Метод printMessage(String message) — вывод сообщения (void)
public static void printMessage(String message) {
    System.out.println(message);
}

// 5. Главный метод — демонстрация работы всех методов
public static void main(String[] args) {

    // Демонстрация sum (два числа)
    int result1 = sum(10, 5);
    printMessage("Сумма 10 + 5 = " + result1);

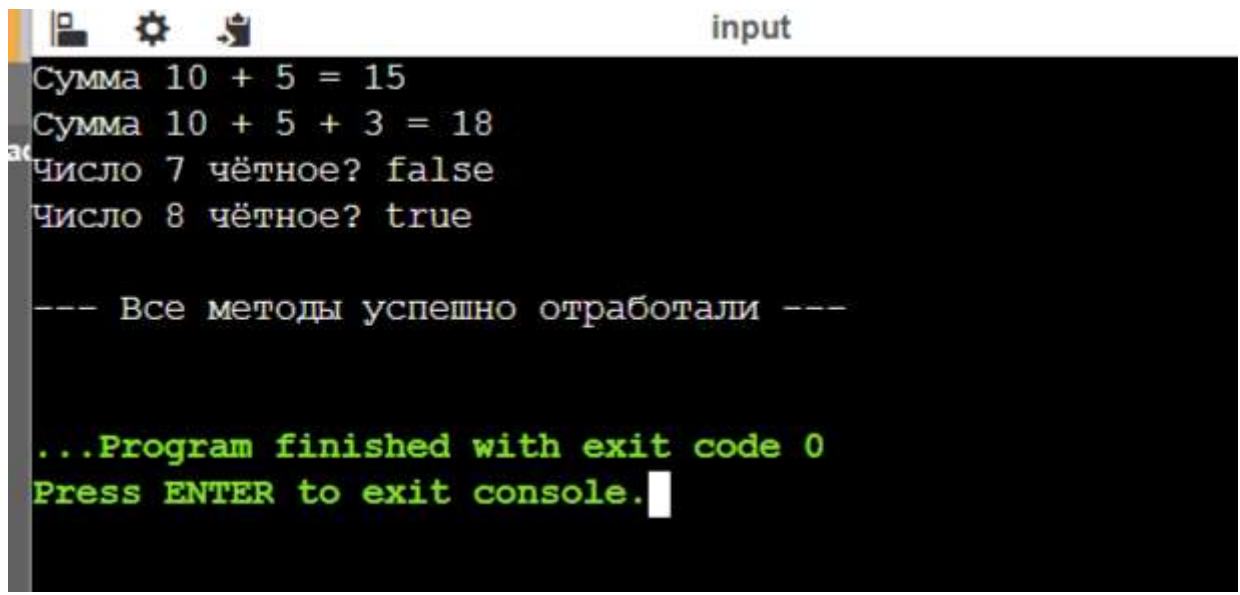
    // Демонстрация перегруженного sum (три числа)
    int result2 = sum(10, 5, 3);
    printMessage("Сумма 10 + 5 + 3 = " + result2);

    // Демонстрация isEven
    int number1 = 7;
    int number2 = 8;
    printMessage("Число " + number1 + " чётное? " + isEven(number1));
    printMessage("Число " + number2 + " чётное? " + isEven(number2));

    // Демонстрация printMessage (сам метод уже используется выше)
    printMessage("\n--- Все методы успешно отработали ---");
}
}

```


Вывод:

A screenshot of a Java IDE's console window. The window has a title bar with standard icons and the word 'input' on the right. The console output is as follows:

```
Сумма 10 + 5 = 15
Сумма 10 + 5 + 3 = 18
Число 7 чётное? false
Число 8 чётное? true

--- Все методы успешно отработали ---

...Program finished with exit code 0
Press ENTER to exit console.
```

ПРАКТИКА №6

Классы и объекты: основы объектно-ориентированного программирования

Программа:

```
public class Main {

    public static void main(String[] args) {

        // 2 объекта по заданию

        Book book1 = new Book("Мастер и Маргарита", "Михаил Булгаков",
1967);

        Book book2 = new Book("Преступление и наказание", "Фёдор
Достоевский", 1866);

        // + 3 дополнительных объекта

        Book book3 = new Book("Война и мир", "Лев Толстой", 1869);
        Book book4 = new Book("Евгений Онегин", "Александр Пушкин", 1833);
        Book book5 = new Book("Тихий Дон", "Михаил Шолохов", 1940);
```

```

// Вывод информации о всех 5 книгах
System.out.println("=== СПИСОК КНИГ ===\n");

    book1.displayInfo();
    book2.displayInfo();
    book3.displayInfo();
    book4.displayInfo();
    book5.displayInfo();
}
}

class Book {
    private String title;
    private String author;
    private int year;

    // Конструктор
    public Book(String title, String author, int year) {
        this.title = title;
        this.author = author;
        this.year = year;
    }

    // Геттеры
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public int getYear() { return year; }
}

```

```
// Сеттеры

public void setTitle(String title) { this.title = title; }
public void setAuthor(String author) { this.author = author; }
public void setYear(int year) { this.year = year; }


// Метод вывода информации
public void displayInfo() {
    System.out.println("Название: " + title);
    System.out.println("Автор: " + author);
    System.out.println("Год издания: " + year);
    System.out.println("-----");
}
}
```

Вывод:

```
==== СПИСОК КНИГ ====
Название: Мастер и Маргарита
Автор: Михаил Булгаков
Год издания: 1967
-----
Название: Преступление и наказание
Автор: Фёдор Достоевский
Год издания: 1866
-----
Название: Война и мир
Автор: Лев Толстой
Год издания: 1869
-----
Название: Евгений Онегин
Автор: Александр Пушкин
Год издания: 1833
-----
Название: Тихий Дон
Автор: Михаил Шолохов
Год издания: 1940
-----

...Program finished with exit code 0
Press ENTER to exit console.
```

ПРАКТИКА №7

Наследование: построение иерархий классов

Программа:

```
public class Main {
    public static void main(String[] args) {
        // Создаём объекты Car и Motorcycle
        Car car = new Car("Toyota", 2020, 4);
        Motorcycle motorcycle = new Motorcycle("Harley-Davidson", 2022);

        // Вызываем метод start() у каждого
        car.start();
        motorcycle.start();
    }
}
```

```
}  
}
```

// 1. Абстрактный суперкласс Vehicle

```
abstract class Vehicle {  
    protected String brand;  
    protected int year;  
  
    // Конструктор  
    public Vehicle(String brand, int year) {  
        this.brand = brand;  
        this.year = year;  
    }  
  
    // Метод start() (будет переопределён в подклассах)  
    public void start() {  
        System.out.println("Транспортное средство заведено");  
    }  
}
```

// 2. Подкласс Car (наследует Vehicle)

```
class Car extends Vehicle {  
    private int numberOfDoors; // дополнительное поле  
  
    // Конструктор  
    public Car(String brand, int year, int numberOfDoors) {  
        super(brand, year); // вызов конструктора суперкласса  
        this.numberOfDoors = numberOfDoors;  
    }  
}
```

```

    }

    // Переопределённый метод start()
    @Override
    public void start() {
        System.out.println("Автомобиль " + brand + " заведён");
    }

    // Геттер для numberOfDoors (опционально)
    public int getNumberOfDoors() {
        return numberOfDoors;
    }
}

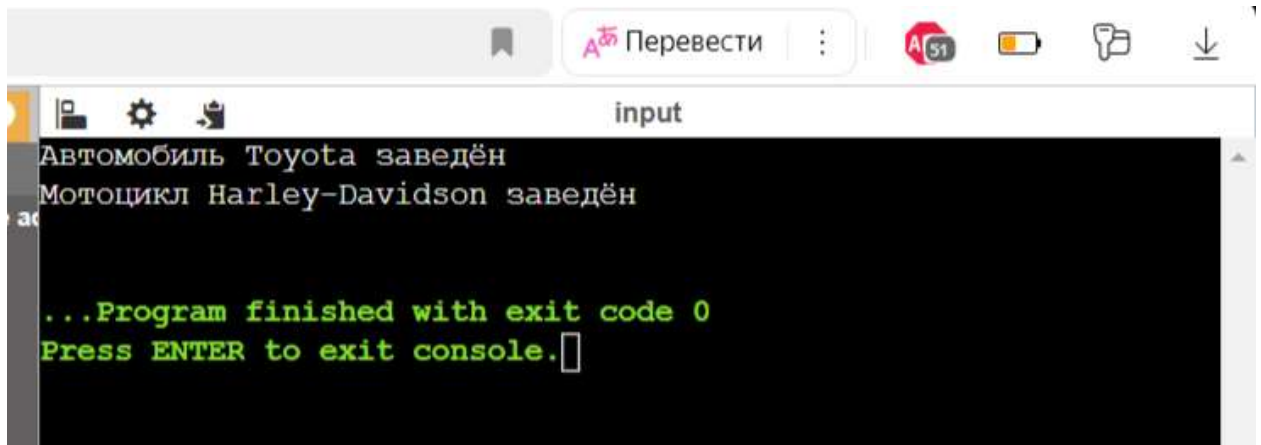
// 3. Подкласс Motorcycle (наследует Vehicle)
class Motorcycle extends Vehicle {

    // Конструктор
    public Motorcycle(String brand, int year) {
        super(brand, year); // вызов конструктора суперкласса
    }

    // Переопределённый метод start()
    @Override
    public void start() {
        System.out.println("Мотоцикл " + brand + " заведён");
    }
}

```

Вывод:

A screenshot of a mobile application interface. At the top, there is a navigation bar with a bookmark icon, a search bar containing the text "Перевести" (Translate), and several status icons including a battery level indicator and a signal strength icon. Below the navigation bar, there is a header area with the word "input" and a settings gear icon. The main content area is a dark-themed console window displaying the following text: "Автомобиль Toyota заведён", "Мотоцикл Harley-Davidson заведён", "...Program finished with exit code 0", and "Press ENTER to exit console." with a cursor at the end of the last line.

ПРАКТИКА №8

Коллекции: работа с динамическими списками (ArrayList)

Программа:

```
import java.util.ArrayList;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
```

```
// 1. Создаём ArrayList<String> для хранения задач
ArrayList<String> tasks = new ArrayList<>();

// 2. Добавляем в список 3 задачи
tasks.add("Купить продукты");
tasks.add("Сделать домашнее задание по Java");
tasks.add("Позвонить родителям");

// 3. Выводим все задачи на экран
System.out.println("=== СПИСОК ЗАДАЧ ===");
System.out.println("Все задачи:");
for (int i = 0; i < tasks.size(); i++) {
    System.out.println((i + 1) + ". " + tasks.get(i));
}
System.out.println("Всего задач: " + tasks.size());
System.out.println();

// 4. Удаляем вторую задачу (индекс 1, так как счёт с 0)
System.out.println("=== УДАЛЕНИЕ ВТОРОЙ ЗАДАЧИ ===");
String removedTask = tasks.remove(1); // удаляем элемент с индексом 1
System.out.println("Удалена задача: " + removedTask);
System.out.println();

// Выводим список после удаления
System.out.println("Список после удаления:");
for (int i = 0; i < tasks.size(); i++) {
    System.out.println((i + 1) + ". " + tasks.get(i));
}
```



```
System.out.println("Всего задач: " + tasks.size());
```

```
System.out.println();
```

```
// 5. Проверяем, есть ли в списке задача "Сделать домашнее задание по Java"
```

```
System.out.println("=== ПРОВЕРКА НАЛИЧИЯ ЗАДАЧИ ===");
```

```
String searchTask = "Сделать домашнее задание по Java";
```

```
if (tasks.contains(searchTask)) {
```

```
    System.out.println("Задача \"" + searchTask + "\" присутствует в списке.");
```

```
    } else {
```

```
        System.out.println("Задача \"" + searchTask + "\" отсутствует в списке.");
```

```
    }
```

```
System.out.println();
```

```
// 6. Сортируем список по алфавиту
```

```
System.out.println("=== СОРТИРОВКА СПИСКА ===");
```

```
Collections.sort(tasks);
```

```
System.out.println("Отсортированный список задач:");
```

```
for (int i = 0; i < tasks.size(); i++) {
```

```
    System.out.println((i + 1) + ". " + tasks.get(i));
```

```
}
```

```
System.out.println("Всего задач: " + tasks.size());
```

```
}
```

```
}
```

Вывод:

```
input
=== СПИСОК ЗАДАЧ ===
Все задачи:
1. Купить продукты
2. Сделать домашнее задание по Java
3. Позвонить родителям
Всего задач: 3

=== УДАЛЕНИЕ ВТОРОЙ ЗАДАЧИ ===
Удалена задача: Сделать домашнее задание по Java

Список после удаления:
1. Купить продукты
2. Позвонить родителям
Всего задач: 2

=== ПРОВЕРКА НАЛИЧИЯ ЗАДАЧИ ===
Задача "Сделать домашнее задание по Java" отсутствует в списке.

=== СОРТИРОВКА СПИСКА ===
Отсортированный список задач:
1. Купить продукты
2. Позвонить родителям
Всего задач: 2

...Program finished with exit code 0
Press ENTER to exit console.
```

ПРАКТИКА №9

Обработка исключений: создание устойчивых программ

Программа:

```
import java.util.InputMismatchException;
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== ПРОГРАММА БЕЗОПАСНОГО ДЕЛЕНИЯ  

        ===\n");

        try {
            // 1. Запрашиваем у пользователя два целых числа
            System.out.print("Введите первое число (делимое): ");
            int num1 = scanner.nextInt();

            System.out.print("Введите второе число (делитель): ");
            int num2 = scanner.nextInt();

            // 2. Выполняем деление первого числа на второе
            int result = num1 / num2;

            System.out.println("\nРезультат: " + num1 + " / " + num2 + " = " + result);

        } catch (InputMismatchException e) {
            // 3.1 Обработка ошибки: введено не число
            System.out.println("\nОшибка: Введено некорректное значение! Нужно  

            ввести целое число.");

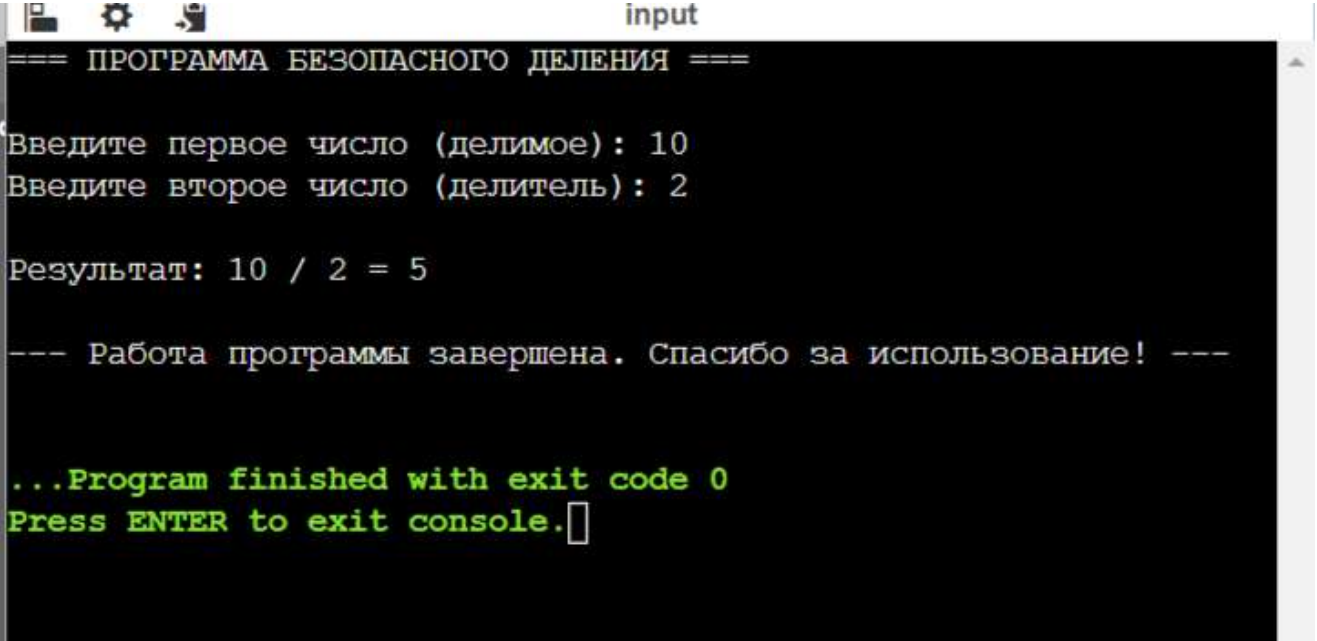
        } catch (ArithmeticException e) {
            // 3.2 Обработка ошибки: деление на ноль
            System.out.println("\nОшибка: Деление на ноль невозможно!");
        }
    }
}

```

```
    } finally {  
        // 4. Блок finally — выполняется всегда  
        System.out.println("\n--- Работа программы завершена. Спасибо за  
использование! ---");  
        scanner.close();  
    }  
}  
}
```

Вывод:

1. Успешное деление



```
input  
=== ПРОГРАММА БЕЗОПАСНОГО ДЕЛЕНИЯ ===  
Введите первое число (делимое): 10  
Введите второе число (делитель): 2  
  
Результат: 10 / 2 = 5  
  
--- Работа программы завершена. Спасибо за использование! ---  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

2. Деление на ноль

```
=== ПРОГРАММА БЕЗОПАСНОГО ДЕЛЕНИЯ ===
Введите первое число (делимое): 10
Введите второе число (делитель): 0

Ошибка: Деление на ноль невозможно!

--- Работа программы завершена. Спасибо за использование! ---

...Program finished with exit code 0
Press ENTER to exit console.
```

3.Ввод текста вместо числа

```
=== ПРОГРАММА БЕЗОПАСНОГО ДЕЛЕНИЯ ===
Введите первое число (делимое): десять
Ошибка: Введено некорректное значение! Нужно ввести целое число.

--- Работа программы завершена. Спасибо за использование! ---

...Program finished with exit code 0
Press ENTER to exit console.
```

ПРАКТИКА №10

Работа с файлами: сохранение и чтение данных

Программа:

```
import java.io.FileWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Arrays;
```

```

import java.util.List;

public class Main {

    // Константа с именем файла
    private static final String FILE_NAME = "students.txt";

    public static void main(String[] args) {

        // 1. Создаём список имён студентов
        List<String> students = Arrays.asList(
            "Иванов Иван",
            "Петрова Мария",
            "Сидоров Алексей",
            "Кузнецова Екатерина",
            "Смирнов Дмитрий"
        );

        System.out.println("=== РАБОТА С ФАЙЛАМИ ===\n");
        System.out.println("Исходный список студентов:");
        for (String student : students) {
            System.out.println(" - " + student);
        }
        System.out.println();

        // 2. Записываем список в файл students.txt (каждое имя с новой строки)
        try (FileWriter writer = new FileWriter(FILE_NAME)) {
            for (String student : students) {

```

```

        writer.write(student + "\n");
    }

    System.out.println("✓ Список студентов успешно записан в файл \"" +
FILE_NAME + "\"");
} catch (IOException e) {

    System.err.println("X Ошибка при записи в файл: " + e.getMessage());
    return; // завершаем программу, если не удалось записать
}

System.out.println();

// 3. Читаем файл и выводим все имена на экран
System.out.println("Чтение из файла \"" + FILE_NAME + "\"");
try {
    Path filePath = Paths.get(FILE_NAME);
    List<String> lines = Files.readAllLines(filePath);

    System.out.println("Список студентов из файла:");
    for (int i = 0; i < lines.size(); i++) {
        System.out.println(" " + (i + 1) + ". " + lines.get(i));
    }

    System.out.println("\n✓ Всего студентов: " + lines.size());

} catch (IOException e) {

    System.err.println("X Ошибка при чтении файла: " + e.getMessage());
}

}
}

```

Вывод:

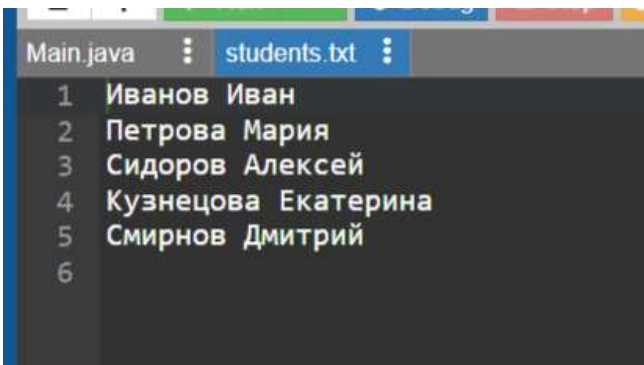
```
--- РАБОТА С ФАЙЛАМИ ---
Исходный список студентов:
- Иванов Иван
- Петрова Мария
- Сидоров Алексей
- Кузнецова Екатерина
- Смирнов Дмитрий

✓ Список студентов успешно записан в файл "students.txt"

Чтение из файла "students.txt":
Список студентов из файла:
1. Иванов Иван
2. Петрова Мария
3. Сидоров Алексей
4. Кузнецова Екатерина
5. Смирнов Дмитрий

✓ Всего студентов: 5

...Program finished with exit code 0
Press ENTER to exit console.
```



```
Main.java  students.txt
1 Иванов Иван
2 Петрова Мария
3 Сидоров Алексей
4 Кузнецова Екатерина
5 Смирнов Дмитрий
6
```

ПРАКТИКА №11

Введение в Stream API: функциональная обработка коллекций

Программа:

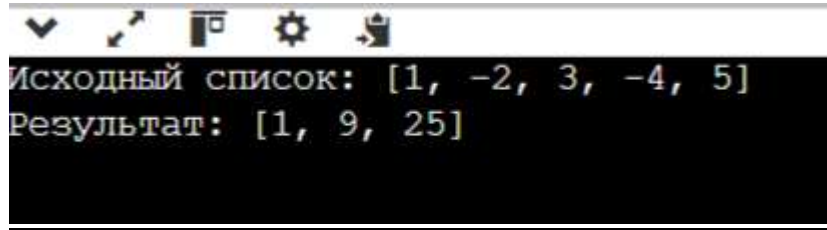
```
import java.util.*;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, -2, 3, -4, 5);

        List<Integer> squaredPositives = numbers.stream()
            .filter(n -> n > 0)
            .map(n -> n * n)
            .collect(Collectors.toList());

        System.out.println("Исходный список: " + numbers);
        System.out.println("Результат: " + squaredPositives);
    }
}
```


Вывод:



```
Исходный список: [1, -2, 3, -4, 5]
Результат: [1, 9, 25]
```

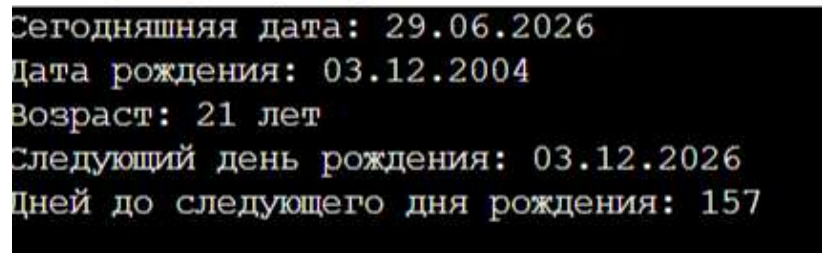
Практика №12

Работа с датой и временем: современный API java.time

Программа:

```
1 import java.time.LocalDate;
2 import java.time.Period;
3 import java.time.format.DateTimeFormatter;
4 import java.time.temporal.ChronoUnit;
5
6 public class Main {
7     public static void main(String[] args) {
8         // 1. Получаем текущую дату
9         LocalDate today = LocalDate.now();
10
11         // 2. Создаем дату рождения (03 декабря 2004 года)
12         LocalDate birthDate = LocalDate.of(2004, 12, 03);
13
14         // 3. Рассчитываем возраст в годах
15         Period age = Period.between(birthDate, today);
16         int years = age.getYears();
17
18         // 4. Определяем, сколько дней осталось до следующего дня рождения
19         LocalDate nextBirthday = birthDate.withYear(today.getYear());
20
21         // Если день рождения уже прошел в этом году, переносим на следующий год
22
23         if (nextBirthday.isBefore(today) || nextBirthday.isEqual(today)) {
24             nextBirthday = nextBirthday.plusYears(1);
25         }
26
27         long daysUntilNextBirthday = ChronoUnit.DAYS.between(today, nextBirthday);
28
29         // 5. Форматируем даты в формат dd.MM.yyyy
30         DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd.MM.yyyy");
31         String formattedToday = today.format(formatter);
32         String formattedBirthDate = birthDate.format(formatter);
33         String formattedNextBirthday = nextBirthday.format(formatter);
34
35         // 6. Выводим все данные
36         System.out.println("Сегодняшняя дата: " + formattedToday);
37         System.out.println("Дата рождения: " + formattedBirthDate);
38         System.out.println("Возраст: " + years + " лет");
39         System.out.println("Следующий день рождения: " + formattedNextBirthday);
40         System.out.println("Дней до следующего дня рождения: " + daysUntilNextBirthday);
41     }
42 }
```

Вывод:



```
Сегодняшняя дата: 29.06.2026
Дата рождения: 03.12.2004
Возраст: 21 лет
Следующий день рождения: 03.12.2026
Дней до следующего дня рождения: 157
```

ПРАКТИКА №13

Работа с коллекциями: основы List, Set и Map

Программа:

```
1 import java.util.ArrayList;
2 import java.util.HashSet;
3 import java.util.HashMap;
4 import java.util.List;
5 import java.util.Set;
6 import java.util.Map;
7
8 public class Main {
9     public static void main(String[] args) {
10         // 1. Создаем список ArrayList с именами студентов (одно имя повторяется дважды)
11         List<String> studentNames = new ArrayList<>();
12         studentNames.add("Диана");
13         studentNames.add("Рома");
14         studentNames.add("Егор");
15         studentNames.add("Рома"); // повторяющееся имя
16         studentNames.add("Виолетта");
17
18         // 2. Выводим список на экран
19         System.out.println("Список студентов (с дубликатами): " + studentNames);
20
21         // 3. Передаем все элементы из ArrayList в HashSet (дубликаты удаляются автоматически)
22
23         Set<String> uniqueNames = new HashSet<>(studentNames);
24         System.out.println("Уникальные имена студентов: " + uniqueNames);
25
26         // 4. Создаем HashMap, где ключ - имя, значение - балл (от 1 до 100)
27         Map<String, Integer> studentScores = new HashMap<>();
28
29         // 5. Заполняем словарь оценками
30         studentScores.put("Диана", 95);
31         studentScores.put("Рома", 78);
32         studentScores.put("Егор", 88);
33         studentScores.put("Виолетта", 92);
34
35         // Выводим результаты на экран
36         System.out.println("\nРезультаты студентов:");
37         System.out.println("Имя | Балл");
38         System.out.println("-----");
39         for (Map.Entry<String, Integer> entry : studentScores.entrySet()) {
40             System.out.println(entry.getKey() + " | " + entry.getValue());
41         }
42     }
43 }
```

Вывод:

```
Список студентов (с дубликатами): [Диана, Рома, Егор, Рома, Виолетта]
Уникальные имена студентов: [Рома, Виолетта, Егор, Диана]

Результаты студентов:
Имя | Балл
-----
Рома | 78
Виолетта | 92
Егор | 88
Диана | 95
```

ПРАКТИКА №14

Многопоточность: основы работы с потоками (Thread и Runnable)

Программа:

```
1 // Класс NumberTask - выполняет числа от 1 до 5 с задержкой 500 мс
2 class NumberTask implements Runnable {
3     @Override
4     public void run() {
5         // Получаем имя текущего потока для наглядности
6         String threadName = Thread.currentThread().getName();
7
8         for (int i = 1; i <= 5; i++) {
9             System.out.println(threadName + " - Число: " + i);
10            try {
11                Thread.sleep(500); // Задержка 500 мс
12            } catch (InterruptedException e) {
13                System.out.println("Поток прерван");
14            }
15        }
16        System.out.println(threadName + " - Завершен");
17    }
18 }
19
20 // Класс LetterTask - выводит буквы от 'A' до 'E' с задержкой 700 мс
21 class LetterTask implements Runnable {
22     @Override
23     public void run() {
24         String threadName = Thread.currentThread().getName();
25
26         // Буквы от 'A' до 'E' (включительно)
27         char[] letters = {'A', 'B', 'B', 'T', 'D', 'E'};
28
29         for (char letter : letters) {
30             System.out.println(threadName + " - Буква: " + letter);
31             try {
32                 Thread.sleep(700); // Задержка 700 мс
33             } catch (InterruptedException e) {
34                 System.out.println("Поток прерван");
35             }
36         }
37         System.out.println(threadName + " - Завершен");
38     }
39 }
40
41 public class Main {
42     public static void main(String[] args) {
43         System.out.println("Главный поток: Запуск программы\n");
44
45         // Создаем задачи
46         NumberTask numberTask = new NumberTask();
47         LetterTask letterTask = new LetterTask();
48
49         // Создаем потоки (задачи выполняются в отдельном потоке)
50         Thread thread1 = new Thread(numberTask, "Поток-Числа");
51         Thread thread2 = new Thread(letterTask, "Поток-Буквы");
52
53         // Запускаем потоки
54         thread1.start();
55         thread2.start();
56
57         System.out.println("Главный поток: Оба потока запущены\n");
58
59         // Ожидаем завершения потоков (опционально)
60         try {
61             thread1.join();
62             thread2.join();
63         } catch (InterruptedException e) {
64             System.out.println("Главный поток прерван");
65         }
66
67         System.out.println("\nГлавный поток: Программа завершена");
68     }
69 }
```

Вывод:

```

Главный поток: Запуск программы

Главный поток: Оба потока запущены

Поток-Буквы - Буква: А
Поток-Числа - Число: 1
Поток-Числа - Число: 2
Поток-Буквы - Буква: Б
Поток-Числа - Число: 3
Поток-Буквы - Буква: В
Поток-Числа - Число: 4
Поток-Числа - Число: 5
Поток-Буквы - Буква: Г
Поток-Числа - Завершен
Поток-Буквы - Буква: Д
Поток-Буквы - Буква: Е
Поток-Буквы - Завершен

Главный поток: Программа завершена

```

ПРАКТИКА №15 и №16

Одномерные и двумерные массивы

Программа:

```

1 import java.util.Random;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println("===== ЗАДАНИЕ 1: Одномерный массив =====\n");
6         task1();
7
8         System.out.println("\n===== ЗАДАНИЕ 2: Двумерный массив (диагональ) =====\n");
9         task2();
10
11         System.out.println("\n===== ЗАДАНИЕ 3: Треугольник Паскаля =====\n");
12         task3();
13     }
14
15     // =====
16     // ЗАДАНИЕ 1: Одномерный массив
17     // Линейный поиск max и min
18     // =====
19     public static void task1() {
20         Random random = new Random();
21         int[] array = new int[10];
22
23         // Заполняем массив случайными числами от 1 до 100
24         for (int i = 0; i < array.length; i++) {
25             array[i] = random.nextInt(100) + 1;
26         }
27
28         // Выводим массив
29         System.out.print("Массив: ");
30         for (int num : array) {
31             System.out.print(num + " ");
32         }
33         System.out.println();
34
35         // Линейный поиск максимального и минимального элементов
36         int min = array[0];
37         int max = array[0];

```



```

38         for (int i = 1; i < array.length; i++) {
39             if (array[i] < min) {
40                 min = array[i];
41             }
42             if (array[i] > max) {
43                 max = array[i];
44             }
45         }
46     }
47
48     System.out.println("Минимальный элемент: " + min);
49     System.out.println("Максимальный элемент: " + max);
50 }
51
52 // =====
53 // ЗАДАНИЕ 2: Двумерный массив (квадратная матрица)
54 // Главная диагональ = 1, остальные = 0
55 // =====
56 public static void task2() {
57     int size = 5; // Размер матрицы 5x5
58     int[][] matrix = new int[size][size];
59
60     // Заполняем матрицу
61     for (int i = 0; i < size; i++) {
62         for (int j = 0; j < size; j++) {
63             if (i == j) {
64                 matrix[i][j] = 1; // Главная диагональ
65             } else {
66                 matrix[i][j] = 0; // Остальные элементы
67             }
68         }
69     }
70
71     // Выводим матрицу
72     System.out.println("Квадратная матрица " + size + "x" + size + ":");
73     for (int i = 0; i < size; i++) {
74         for (int j = 0; j < size; j++) {
75             System.out.print(matrix[i][j] + " ");

```

```

76         }
77         System.out.println();
78     }
79 }
80
81 // =====
82 // ЗАДАНИЕ 3: Треугольник Паскаля (ступенчатый массив)
83 // =====
84 public static void task3() {
85     int rows = 6; // Количество строк
86     int[][] pascal = new int[rows][];
87
88     // Заполняем треугольник Паскаля
89     for (int i = 0; i < rows; i++) {
90         pascal[i] = new int[i + 1]; // Каждая строка длиннее на 1
91         pascal[i][0] = 1; // Первый элемент каждой строки = 1
92         pascal[i][i] = 1; // Последний элемент каждой строки = 1
93
94         // Заполняем внутренние элементы
95         for (int j = 1; j < i; j++) {
96             pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
97         }
98     }
99
100     // Выводим треугольник Паскаля (отформатированный)
101     System.out.println("Треугольник Паскаля (" + rows + " строк):");
102     for (int i = 0; i < rows; i++) {
103         // Отступы для красивого вывода
104         for (int k = 0; k < rows - i - 1; k++) {
105             System.out.print(" ");
106         }
107         for (int j = 0; j < pascal[i].length; j++) {
108             System.out.print(pascal[i][j] + " ");
109         }
110         System.out.println();
111     }
112 }
113 }

```

Вывод:

```
===== ЗАДАНИЕ 1: Одномерный массив =====

Массив: 94 57 50 4 46 61 76 47 43 29
Минимальный элемент: 4
Максимальный элемент: 94

===== ЗАДАНИЕ 2: Двумерный массив (диагональ) =====

Квадратная матрица 5x5:
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1

===== ЗАДАНИЕ 3: Треугольник Паскаля =====

Треугольник Паскаля (6 строк):
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
```

ПРАКТИКА №17

Хеш-таблицы и ассоциативные массивы

Программа:

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class Main {
5     public static void main(String[] args) {
6         // 1. Создаем коллекцию HashMap<String, Integer>
7         HashMap<String, Integer> journal = new HashMap<>();
8
9         // 2. Заполняем журнал данными пяти студентов (метод put)
10        journal.put("Виктория", 95);
11        journal.put("Глеб", 78);
12        journal.put("Екатерина", 88);
13        journal.put("Дмитрий", 92);
14        journal.put("Дарья", 67);
15
16        System.out.println("Журнал успеваемости:");
17        System.out.println(journal);
18        System.out.println();
19
20        // 3. Выводим оценку конкретного студента (метод get)
21        String studentName = "Виктория";
22        Integer score = journal.get(studentName);
23        System.out.println("Оценка студента " + studentName + ": " + score);
```

```

24     System.out.println();
25
26     // 4. Проверим наличие студента по имени (метод containsKey)
27     String checkName = "Петр";
28     if (journal.containsKey(checkName)) {
29         System.out.println("Студент " + checkName + " есть в журнале");
30     } else {
31         System.out.println("Студента " + checkName + " НЕТ в журнале");
32     }
33     System.out.println();
34
35     // 5. Удаляем одного студента (метод remove) и выводим оставшийся список
36     String removeName = "Дарья";
37     journal.remove(removeName);
38     System.out.println("После удаления студента " + removeName + ":");
39
40     // Выводим весь оставшийся список
41     System.out.println("Оставшиеся студенты:");
42     for (Map.Entry<String, Integer> entry : journal.entrySet()) {
43         System.out.println(entry.getKey() + " - " + entry.getValue() + " баллов");
44     }
45 }
46 }

```

Вывод:

Журнал успеваемости:
 {Дарья=67, Екатерина=88, Виктория=95, Дмитрий=92, Глеб=78}

Оценка студента Виктория: 95

Студента Петр НЕТ в журнале

После удаления студента Дарья:
 Оставшиеся студенты:
 Екатерина – 88 баллов
 Виктория – 95 баллов
 Дмитрий – 92 баллов
 Глеб – 78 баллов

ПРАКТИКА №18

Матричные вычисления и обработка табличных данных

Программа:

```

1 public class Main {
2     public static void main(String[] args) {
3         // 1. Создаем и инициализируем двумерный массив 3x3
4         int[][] matrix = {
5             {5, 12, 8},
6             {3, 7, 14},
7             {6, 9, 11}
8         };
9
10        // Выводим матрицу для наглядности
11        System.out.println("Матрица 3x3:");
12        for (int i = 0; i < 3; i++) {
13            for (int j = 0; j < 3; j++) {
14                System.out.print(matrix[i][j] + " ");
15            }
16            System.out.println();
17        }
18        System.out.println();
19
20        // 2. Объявляем переменную-аккумулятор
21        int sum = 0;
22
23        // 3-4. Вложенный цикл для обхода всех ячеек и суммирования
24        for (int i = 0; i < matrix.length; i++) { // строки
25            for (int j = 0; j < matrix[i].length; j++) { // столбцы
26                sum += matrix[i][j]; // прибавляем значение ячейки
27            }
28        }
29
30        // 5. Выводим итоговую сумму
31        System.out.println("Сумма всех элементов матрицы: " + sum);
32    }
33 }

```

Вывод:

```

Матрица 3x3:
5 12 8
3 7 14
6 9 11

Сумма всех элементов матрицы: 75

```